Model Predictive Control

Silvan Stadelmann - 4. Februar 2026 - v1.1.1

github.com/silvasta/summary-mpc

Contents

Requirements and Steps to MPC

1. **Model of the System** dynamics to state space
2. **State Estimator** track trajectory and disturbance
3. **Optimal Control Problem** define control strategy
4. **Optimization problem** mathematical formulation
5. **Get Optimal Control Sequence** solve op in time
6. **Verify Closed-Loop Performance** iterative tests

1 Nominal MPC

MPC Mathematical Formulation

$$\underset{U}{\operatorname{argmin}} \sum_{i=0}^{N-1} I(x_i, u_i) + I_f(x_N) \tag{1}$$

Constraints
$$\begin{aligned} x_0 &= x(k) \\ x_{i+1} &= f(x_i, u_i) \quad \text{linear case : } Ax_i + Bu_i \\ x_i &\in \mathcal{X} \\ u_i &\in \mathcal{U} \\ x_N &\in \mathcal{X}_f \end{aligned}$$

$I_f(\cdot)$ and $\mathcal{X}_f$ are chosen to mimic an infinite horizon.

What can go wrong with *standard* MPC?

- No feasibility guarantee, the problem may not have a solution
- No stability guarantee, trajectories may not converge to origin

Stability of MPC - Main Result

Assumptions

1. Stage cost is strictly positive and only zero at the origin
2. Terminal set is **invariant** under local control law $\kappa_f(x_i)$:

$$x_{i+1} = Ax_i + B\kappa_f(x_i) \in \mathcal{X}_f \ \forall\, x_i \in \mathcal{X}_f$$

All state and input **constraints are satisfied** in $\mathcal{X}_f$:

$$\mathcal{X}_f \in X, \kappa_f(x_i) \in U \ \forall\, x_i \in \mathcal{X}_f$$

3. Terminal cost is a continuous **Lyapunov function** s.t.

$$l_f(x_{i+1}) - l_f(x_i) \leq -l(x_i, \kappa_f(x_i)) \quad \forall x_i \in \mathcal{X}_f$$

Theorem 1. Under the previous assumptions, the closed-loop system under the MPC control law $u_0^\star(x)$ is asymptotically stable and the set $\mathcal{X}_f$ is positive invariant for

$$x(k+1) = Ax(k) + Bu_0^\star(x(k))$$

Same thing for non-linear MPC! (f/g(x,u) instead of A/B)

Finite-horizon MPC may not satisfy constraints for all time!

Finite-horizon MPC may not be stable!

- An infinite-horizon provides stability and invariance.
- Infinite-horizon *faked* by forcing final state into an invariant set for which there exists invariance-inducing controller, whose infinite-horizon cost can be expressed in closed-form.
- Extends to non-linear systems, but compute sets is difficult!

2 Practical MPC

2.1 Steady-state Target Problem

- Reference is achieved by the target state x_s if $z_s = Hx_s = r$
- Target state should be a steady-state, i.e. $x_s = Ax_s + Bu_s$

$$\begin{aligned} x_s &= Ax_s + Bu_s \\ z_s &= Hx_s = r \end{aligned} \iff \begin{bmatrix} \mathbb{I} - A & -B \\ H & 0 \end{bmatrix} \begin{bmatrix} x_s \\ u_s \end{bmatrix} = \begin{bmatrix} 0 \\ r \end{bmatrix}$$

$\nexists$ solution $\rightarrow \min (Hx_s - r)^\top Q_s (Hx_s - r)$ (closest x to r)

If $\exists$ multiple feasible $u_s \rightarrow$ compute cheapest: $\min u_s^\top R_s u_s$

$$\min_U |z_N - Hx_s|^2_{P_z} + \sum_{i=1}^{N-1} |z_i - Hx_s|^2_{Q_z} + |u_i - u_s|^2_R$$

2.2 Offset-free Reference Tracking

Reference Tracking

$$\begin{aligned} \Delta x_{k+1} &= x_{k+1} - x_s \\ &= A\Delta x_k + Bu_k - (Ax_s + Bu_s) \\ &= A\Delta x_k + B\Delta u_k \end{aligned}$$

$$\begin{aligned} G_x x \leq h_x \\ G_u u \leq h_u \end{aligned} \Rightarrow \begin{aligned} G_x \Delta x \leq h_x - G_x x_s \\ G_u \Delta u \leq h_u - G_u u_s \end{aligned}$$

.

Convergence

Assume feasible target with $x_s \in \mathcal{X}, u_s \in \mathcal{U}$, choose terminal weight $V_f(x)$ and constraint $\mathcal{X}_f$ as in regulation case satisfying

$$V_f(x(k+1)) - V_f(x(k)) \leq -I(x(k), Kx(k))$$

and $(A + BK)x \in \mathcal{X} \quad \forall x \in \mathcal{X}_f$ for both

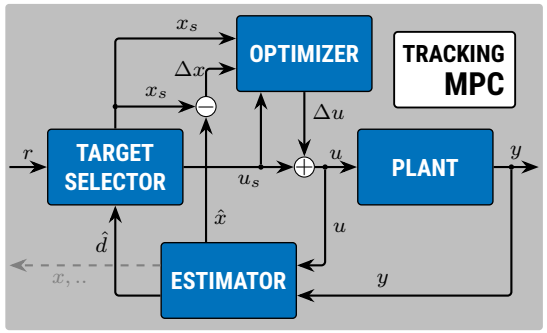
If in addition the target reference x_s, u_s is such that

$$x_s \oplus \mathcal{X}_f \subseteq \mathcal{X}, K\Delta x + u_s \in \mathcal{U}, \quad \forall \Delta x \in \mathcal{X}_f$$

then the closed loop system converges to the target reference.

Proof. Invariance under local control law is inherited from regulation case. Constraint satisfaction is provided by extra conditions and convergence comes from the asymptotic stability of the regulation problem: $\Delta x(k) \rightarrow 0$ for $k \rightarrow \infty$ □

Terminal set use $\mathcal{X}_f^{\text{scaled}} = \alpha \mathcal{X}_f$ (s.t. constraints satisfied)



Disturbance Cancellation

Approach Model the disturbance, use the measurements and model to estimate the state and disturbance and find control inputs that use the disturbance estimate to remove offset.

Augmented Model

$$\begin{aligned} x_{k+1} &= Ax_k + Bu_k + B_d d_k \\ y_k &= Cx_k + C_d d_k \end{aligned}$$

Constant disturbance $d_{k+1} = d_k$

Observable iff $\begin{bmatrix} A-I & B_d \\ C & C_d \end{bmatrix}$ has full rank (assuming $n_x = n_d$)

Observer For Augmented Model

$$\begin{bmatrix} \hat{x}_{k+1} \\ \hat{d}_{k+1} \end{bmatrix} = \begin{bmatrix} A & B_d \\ 0 & \mathbb{I} \end{bmatrix} \begin{bmatrix} \hat{x}_k \\ \hat{d}_k \end{bmatrix} + \begin{bmatrix} B \\ 0 \end{bmatrix} u_k + \begin{bmatrix} L_x \\ L_d \end{bmatrix} (C\hat{x}_k + C_d\hat{d}_k - y_k)$$

Error Dynamics $\Rightarrow$ choose L s.t error dynamics converge to 0

$$\begin{bmatrix} x_{k+1} - \hat{x}_{k+1} \\ d_{k+1} - \hat{d}_{k+1} \end{bmatrix} = \left(\begin{bmatrix} A & B_d \\ 0 & \mathbb{I} \end{bmatrix} + \begin{bmatrix} L_x \\ L_d \end{bmatrix} \begin{bmatrix} C & C_d \end{bmatrix} \right) \begin{bmatrix} x_k - \hat{x}_k \\ d_k - \hat{d}_k \end{bmatrix}$$

Lemma 1. Steady-state of an asym. stable observer satisfies:

$$\begin{bmatrix} A - \mathbb{I} & B \\ C & 0 \end{bmatrix} \begin{bmatrix} \hat{x}_\infty \\ u_\infty \end{bmatrix} = \begin{bmatrix} -B_d \hat{d}_\infty \\ y_\infty - C_d \hat{d}_\infty \end{bmatrix} \text{ (for } n_y = n_d)$$

$\Rightarrow$ Observer output $C\hat{x}_\infty + C_d\hat{d}_\infty$ tracks y_∞ without offset

Reference Tracking with Disturbance Cancellation

Goal Track constant reference: $Hy(k) = z(k) \rightarrow r, k \rightarrow \infty$

$$\begin{aligned} x_s &= Ax_s + Bu_s + B_d\hat{d}_\infty \\ z_s &= H(Cx_s + C_d\hat{d}_\infty) = r \end{aligned}$$

$$\begin{bmatrix} A - I & B \\ HC & 0 \end{bmatrix} \begin{bmatrix} x_s \\ u_s \end{bmatrix} = \begin{bmatrix} 0 \\ r - HC_d\hat{d} \end{bmatrix}$$

Offset-free Tracking - Main Result

Theorem 2. Assuming RHC recursively feasible, $n_d = n_y$, unconstrained for $k \geq j$, and the closed loop system

$$\begin{aligned} x(k+1) &= Ax(k) + B\kappa(\cdot) + B_d d \ \text{ with } (\cdot) = (\hat{x}, \hat{d}, r) \\ \hat{x}(k+1) &= (A + L_x C)\hat{x}(k) + (B_d + L_x C_d)\hat{d}(k) \\ &\quad + B\kappa(\cdot) - L_x y(k) \\ \hat{d}(k+1) &= L_d C\hat{x}(k) + (\mathbb{I} + L_d C_d)\hat{d}(k) - L_d y(k) \end{aligned}$$

converges, then $z(k) = Hy(k) \rightarrow r$ as $k \rightarrow \infty$

2.3 Soft Constraints

Input constraints are dictated by physical constraints on the actuators and are **usually hard**

State/output constraints arise from practical restrictions on the allowed operating range and are **rarely hard**

Hard state/output constraints always lead to **complications in the controller implementation**

Soft Constrained MPC

$$\min_{uEPS} \sum_{i=0}^{N-1} x_i^\top Q x_i + u_i^\top R u_i + l_\epsilon(\epsilon_i) + x_N^\top P x_i + l_\epsilon(\epsilon_N) \quad (2)$$

Quadratic penalty
$l_\epsilon(\epsilon_i) = \epsilon_i^\top S \epsilon_i$
(e.g $S = Q$)
+ linear norm penalty
$l_\epsilon(\epsilon_i) + = v|\epsilon_i|_{1/\infty}$

Constraints
$x_{i+1} = Ax_i + Bu_i$
$H_x x_i \le k_x + \epsilon_i$
$H_u u_i \le k_u$
$\epsilon_i \ge 0$ slack variable

Original $\min_z f(z)$ s.t. $g(z) \le 0$

Softened $\min_{z,\epsilon} f(z) + l_\epsilon(\epsilon)$ s.t. $g(z) \le \epsilon$, $\epsilon \ge 0$

Requirement on $l_\epsilon(\epsilon)$ If the original problem has a feasible solution $z^\star$, then the softened problem should have the same solution $z^\star$, and $\epsilon = 0$.
Theorem 3 (Exact Penalty Funtcion). $l_\epsilon(\epsilon) = v \cdot \epsilon$ satisfies requirement for any $v > \lambda^\star \ge 0$, where $\lambda^\star$ is optimal Lagrange multiplier for original problem

3 Robust MPC

Uncertain System

$$x(k+1) = g(x(k), u(k), w(k); \theta)$$

Min-Max Robust Formulation

$$\min_U \max_W \sum_{i=1}^{N} \ell(x_{i|k}, u_{i|k})$$
$$\text{s.t.} \quad x_{i+1|k} = Ax_{i|k} + Bu_{i|k} + w_{i|k},$$
$$\forall w_{i|k} \in \mathcal{W} \quad x_{i|k} \in \mathcal{X} \quad u_{i|k} \in \mathcal{U}$$

Robust Constraint Satisfaction
Idea Compute a set of tighter constraints such that if the nominal system meets these constraints, then the uncertain system will too. We then do MPC on the nominal system.

Goal Ensure constraints satisfied for the MPC sequence.

Disturbance reachable set $\mathcal{F}_i = \bigoplus_{j=0}^{i-1} A^j \mathcal{W}$

Robust Open-Loop MPC

$$\min_U \sum_{i=0}^{N-1} l(x_i, u_i) + l_f(x_N) \quad \text{s.t. } x_{i+1} = Ax_i + Bu_i$$
$$x_0 = x(k)$$
$$x_i \in \mathcal{X} \ominus \mathcal{F}_i$$
$$u_i \in \mathcal{U}$$
$$x_N \in \mathcal{X}_f \ominus \mathcal{F}_N$$

Closed Loop Robust MPC
Idea Separate the available control authority into two parts:

$$z(k+1) = Az(k) + Bv(k)$$

steers noise-free *nominal* system to origin

$$u_i = K(x_i - z_i) + v_i$$

compensates for deviations, i.e. a *tracking* controller, to keep the real trajectory close to the nominal system. ⇒ We fix the linear feedback controller K offline, and optimize over the nominal inputs $\{v_0, ..., v_{N-1}\}$ and nominal trajectory $\{z_0, ..., z_N\}$, which results in a convex problem.

$$e_{i+1} = x_{i+1} - z_{i+1} = (A + BK)e_i + w_i$$

3.1 Robust Constraint-Tightening MPC

Robust Constraint-Tightening MPC

$$\min_{Z,V} \sum_{i=0}^{N-1} l(z_i, v_i) + l_f(z_N) \quad \text{subj. to } z_{i+1} = Az_i + Bv_i$$
$$z_0 = x(k)$$
$$z_i \in \mathcal{X} \ominus \mathcal{F}_i$$
$$u_i \in \mathcal{U} \ominus K\mathcal{F}_i$$
$$z_N \in \mathcal{X}_f \ominus \mathcal{F}_N$$

$\mathcal{F}_0 := 0 \quad F_i := \mathcal{W} \oplus A_K\mathcal{W} \oplus \dots A_K^{i-1}\mathcal{W},$

Control Law $u(k) = v_0^\star + K(x(k) - z_0) = v_0^\star$

3.2 Robust Tube MPC

Idea Ignore noise and plan the nominal trajectory, bound maximum error at any time with RPI set $\mathcal{E} : \epsilon_i \in \mathcal{E} \epsilon_{i+1} \in \mathcal{E}$

Ideally $\mathcal{E}$ is selected as the minimum RPI set F_∞

We know that the real trajectory stays 'nearby' the nominal one $x_i \in z_i \oplus \mathcal{E}$ because we plan to apply the controller $u_i = K(x_i - z_i) + v_i)$ in the future.

(we won't actually do this, but it's a valid sub-optimal plan)

We must ensure that all possible state trajectories satisfy the constraints This is now equivalent to ensuring that $x_i \in z_i \oplus \mathcal{E}$

What do we need to make this work?

Compute the set $\mathcal{E}$ that the error will remain inside

Previously we wanted the **maximum robust invariant set**, or the largest set in which our terminal control law works.

We now want the **minimum robust invariant set**, or the smallest set that the state will remain inside despite the noise.

Modify constraints on nominal trajectory $\{z_i\}$

$x_i \in z_i \oplus \mathcal{E} = \{z_i + e | e \in \mathcal{E}\}$

Tube MPC

Cost function $J(Z, V) := \sum_{i=0}^{N-1} l(z_i, v_i) + l_f(z_N)$

Feasible set
$$\mathcal{Z}(x_0) := \begin{cases} z_{i+1} & = Az_i + Bv_i \\ z_i & \in \mathcal{X} \ominus \mathcal{E} \\ v_i & \in \mathcal{U} \ominus K\mathcal{E} \\ z_N & \in \mathcal{X}_f \\ x_0 & \in z_0 \oplus \mathcal{E} \end{cases}$$

Optimization $(V^\star(x_0), Z^\star(x_0)) = \text{argmin}_{V,Z}\{J(Z,V) | (Z,V) \in \mathcal{Z}(x_0)\}$

Control law $\mu_{\text{tube}}(x) := K(x - z_0^\star(x)) + v_0^\star(x)$

Theorem 4 (Robust Invariance of Tube MPC). The set $\mathcal{Z} := \{x | \mathcal{Z}(x) \neq \emptyset\}$ is a robust invariant set of the system $x(k+1) = Ax(k) + B\mu_{\text{tube}}(x(k)) + w(k)$ subject to the constraints $x, u \in \mathcal{X} \times \mathcal{U}$.
Theorem 5 (Robust Stability of Tube MPC). The state $x(k)$ of the system $x(k+1) = Ax(k) + B\mu_{\text{tube}}(x(k)) + w(k)$ converges to the limit of the set $\mathcal{E}$.

Tube MPC - Quick Summary

To implement tube MPC:

– Offline –

1. Stabilizing controller K so that $A + BK$ is (Schur) stable
2. Compute the minimal robust invariant set $E = F_\infty$ for the system $x(k+1) = (A + BK)x(k) + w(k), w \in \mathcal{W}^1$
3. Compute tightened constraints $\bar{\mathcal{X}} := \mathcal{X} \ominus \mathcal{E}, \bar{\mathcal{U}} := \mathcal{U} \ominus K\mathcal{E}$
4. Choose terminal weight function l_f and constraint $\mathcal{X}_f$ satisfying assumptions*

– Online –

1. Measure / estimate state x
2. Solve optimization problem for $(V^\star(x_0), Z^\star(x_0))$
3. Set the input to $u = K(x - z_0^\star(x)) + v_0^\star(x)$

4 Stochastic MPC

Stochastic MPC extends deterministic MPC to handle uncertainties modeled as random variables, such as additive noise or parameter variations. A core feature is replacing hard constraints with chance constraints to account for uncertainty.

$$\mathbb{P}(x_k \in \mathcal{X}) \ge 1 - \epsilon, \quad \mathbb{P}(u_k \in \mathcal{U}) \ge 1 - \delta,$$

This softens constraints, improving feasibility over robust MPC.

Mean Dynamics $\bar{x}_{k+1} = A\bar{x}_k + B\bar{u}_k$ (Deterministic)

Variance Dynamics $\Sigma_{k+1} = (A+BK)\Sigma_k(A+BK)^\top + \Sigma_w$

The variance Σ_k can be precomputed offline as it does not depend on the actual state measurements.

Stochastic MPC

$$J = \mathbb{E}\left[\|x_N\|_P^2 + \sum_{k=0}^{N-1} \|x_k\|_Q^2 + \|u_k\|_R^2\right]$$

Using the property $\mathbb{E}[x^\top Q x] = \bar{x}^\top Q \bar{x} + \text{tr}(Q\text{var}(x))$, the cost decomposes into:
Nominal Cost A standard quadratic cost on the mean trajectories $(\bar{x}, \bar{u})$.
Constant Offset A term involving $\text{tr}(Q\Sigma_k)$ which depends only on the noise statistics and the gain K.
Conclusion For a fixed K, optimizing the expected cost is equivalent to optimizing the nominal cost of the mean system.

System dynamics $x_{i+1} = Ax_i + Bu_i + w_i$ (probabilistic)
Chance constraints $\mathbb{P}(x_i \in \mathcal{X}, u_i \in \mathcal{U}) \ge 1 - \epsilon$
Initial condition $x_0 = x_k$
Terminal set $x_N \in \mathcal{X}_f$ (probabilistically)

Tube-like Feedback Policy (to control spread of trajectories)

$$u(k) = \bar{u}_k + K(x(k) - \bar{x}_k)$$

For tractability, approximations like affine feedback policies $u_i = Kx_i + v_i$ (where v_i is optimized) are used to close the loop in predictions.

For general distributions or polytopic sets, we use

Probabilistic Reachable Set $\mathcal{F}_k^p$ such that $\Pr(e(k) \in \mathcal{F}_k^p) \ge p$

The constraint is satisfied if: $\bar{x}_k \in \mathcal{X} \ominus \mathcal{F}_k^p$

Feasibility
Large (unbounded) disturbances can drive the system into regions where the optimization problem becomes infeasible at the next time step.

Asymptotic Average Performance bound
Stability definition $\lim_{T\to\infty} \frac{1}{T} \sum \mathbb{E}[l(x,u)] \le \text{tr}(P\Sigma_w)$

Recovery Initialization
If the measured state $x(k)$ is infeasible for the MPC, the solver is initialized with the **predicted mean** from the previous step $(\bar{x}_{1|k-1})$ instead of the actual measurement. This ensures recursive feasibility but sacrifices performance.

Indirect Feedback SMPC
A more advanced formulation where the nominal state $z(k)$ evolves independently of the measurement $x(k)$ in the constraints, but $x(k)$ enters the **cost function**.

- Pro Guarantees recursive feasibility and satisfaction of closed-loop chance constraints.

- Con The feedback is "weaker" than direct state-feedback MPC.

5 Nonlinear MPC

5.1 Optimization

Because NMPC requires solving NLPs in real-time, algorithmic efficiency is critical.

Sequential Quadratic Programming (SQP) The standard method for NMPC. It solves a sequence of Quadratic Programs (QP) that approximate the NLP locally using a quadratic cost and linearized constraints.

Sequential vs. Simultaneous Methods

Sequential (Single-shooting) Only control inputs are decision variables, states are eliminated via simulation. Results in small, dense QPs.

Simultaneous (Multiple-shooting) Both states and inputs are decision variables. Results in large, sparse QPs. (Preferred in practice for better convergence and handling of unstable dynamics)

Real-Time Iteration (RTI) To meet strict timing constraints, RTI performs only one SQP iteration per sampling time, using warm-starts from the previous step.

5.2 Robust NMPC

In linear systems, the error dynamics $e = x - z$ are independent of the nominal state. In nonlinear systems, the **error dynamics** are **state-dependent**, making robust design much harder.
Incremental Stability
Incremental Lyapunov function $V_\delta(x, z)$ such that:

$$V_\delta(f(x,u), f(z,v)) \le \rho V_\delta(x, z)$$

This characterizes how "close" the real trajectory x stays to the nominal trajectory z.

Nonlinear Homothetic Tubes The tube is defined as the level set of the incremental Lyapunov function: $\mathbb{X}_i = \{x \mid V_\delta(x, z_i) \le \delta_i\}$. The tube size δ_i is propagated dynamically:

$$\delta_{i+1} = \tilde{\rho}\delta_i + \bar{w}(z_i, v_i)$$

where $\bar{w}$ accounts for the worst-case model mismatch/disturbance.

Constraint Tightening Nominal constraints are tightened based on the current tube size δ_i, ensuring that if the nominal state

is in the tightened set, the real state is guaranteed to satisfy original constraints.

5.3 Stochastic NMPC

SNMPC aims for **PRS** rather than worst-case tubes.

The constraints are tightened using the PRS. The cost function is typically the **expected cost** conditioned on the current state.

6 Parameter Estimation Learning Based MPC

The fundamental goal of LBMPC is to improve performance by learning unknown system dynamics $g(x, u)$ online while maintaining the stability and safety guarantees of robust MPC.

6.1 Regression Techniques for Model Learning

Before controlling the system, one must quantify the uncertainty.

Parametric vs. Non-parametric Parametric models (like BLR) use a finite set of weights θ. Non-parametric models (like GPs) use the data points themselves.

Robust vs. Stochastic Robust methods (Set-Membership) provide hard bounds. Stochastic methods (BLR, GP) provide probability distributions.

1. **Bayesian Linear Regression (BLR)** Assumes $y = \Phi(x)^\top \theta + w$. It updates a Gaussian posterior $p(\theta|\mathcal{D}) = \mathcal{N}(\mu_\theta, \Sigma_\theta)$ using Bayes' rule.
2. **Gaussian Processes (GP)** A non-parametric method defined by a mean $m(x)$ and a kernel $k(x, x')$. It provides a mean prediction $\hat{f}(x_*)$ and a variance var$[f(x_*)]$.
3. **Set-Membership (SM) Estimation** A robust approach. Given a bound on noise $w \in \mathcal{W}$, it computes a **non-falsified set** Δ_i of parameters consistent with the data. The parameter set Ω_i is updated via intersection: $\Omega_i = \Omega_{i-1} \cap \Delta_i$.

6.2 Parameter Estimation Methods

System Modeling

For parameter θ and with w_k, v_k disturbances/noise:

$$x_{k+1} = f(x_k, u_k, \theta) + w_k, \quad y_k = h(x_k, \theta) + v_k$$

Estimation refines θ using observed data. Divided into offline (pre-computed) and online (real-time) approaches.

Offline Estimation Uses historical data for initial θ fitting.

Least Squares (LS): Minimizes residual sum:

$$\hat{\theta} = \arg\min_\theta \sum_{i=1}^{N} \|y_i - \phi_i^T \theta\|^2$$

where ϕ_i is the regressor vector.

Extensions: Recursive LS (RLS) for incremental updates, Bayesian methods for uncertainty quantification.

Online Estimation Adapts θ during operation, enabling learning.

Kalman Filter (KF)-based: Treats θ as augmented state, updating via prediction-correction.

$$\hat{\theta}_{k|k} = \hat{\theta}_{k|k-1} + K_k(y_k - \hat{y}_{k|k-1})$$

where K_k is Kalman gain.

Machine Learning Integration: Gaussian Processes (GP) or Neural Networks (NN) for nonparametric estimation, capturing complex θ dynamics. GP models uncertainty as:

$$f(\cdot) \sim \mathcal{GP}(m(\cdot), k(\cdot, \cdot))$$

with mean m and kernel k.

6.3 Robust Performance LBMPC

Core objective is to **decouple** performance from safety.

The controller uses two models simultaneously:

Nominal Linear Model

$$z_{i+1} = Az_i + Bu_i$$

Used to ensure safety. Constraints are tightened relative to this model using Robust MPC techniques (e.g., tube-based MPC).

Learned Nonlinear Model

$$x_{i+1} = Ax_i + Bu_i + \mathcal{O}_k(x_i, ui)$$

The *Oracle* $\mathcal{O}_k$ represents the learned dynamics. The MPC **cost function** is evaluated using this model to achieve high performance.

Result Even if the learned model is wrong, the nominal model ensures the system remains within the *safe* tube.

6.4 Robust Adaptive MPC (Set-Membership Update)

While the Performance model can be any black box, **Uncertainty Set** $\mathcal{W}$ must be updated carefully to avoid losing feasibility.

Mechanism As new data arrives, the set Ω shrinks.

Fixed Shape Parameter Set To prevent the number of constraints from growing to infinity, the set Ω is over-approximated by a polytope of fixed shape, where only the offsets h are updated via a Linear Program (LP).

Guarantee Because $\Omega_{i+1} \subseteq \Omega_i$, the feasible region of the MPC is non-decreasing ($X^{i+1}_{feas} \supseteq X^i_{feas}$), ensuring **recursive feasibility**.

6.5 Stochastic Learning-based MPC (Cautious MPC)

When using GPs, the uncertainty is propagated through the horizon.

Uncertainty Propagation Since propagating a Gaussian through a nonlinear GP is hard, approximations are used:

Mean Equivalent Treat GP input as deterministic at its mean.

Taylor Expansion Linearize the GP mean around the current state mean.

Exact Moment Matching Analytically compute mean and variance (possible for RBF kernels).

Cautious MPC The controller is *cautious* because the variance $\Sigma(x)$ appears in the constraints (chance constraints) and the cost function.

7 Safety Filter

Learning-based controllers (like Reinforcement Learning) or human operators are excellent at exploring complex tasks but often lack formal safety guarantees.
The idea of **Safety Filters** is to filter the nominal input $u^\star$ and produce a safe input u that minimally deviates from $u^\star$ while ensuring the system state remains in a safe set.

7.1 Decoupling

Instead of building safety directly into a complex objective function, we decouple the problem.

1. **Performance** A controller (e.g. a Learning-based Controller, human or AI input,...) focuses on the complex objectives and provides a suggested input $u_L(k)$.
2. **Safety Filter** A verification module that sits between the controller and the system. It verifies $u_L(k)$ and modifies it **minimally** only if required to ensure constraint satisfaction for all future times.

7.2 Safe Set

A central concept is the **Safe Set** $\mathcal{S}$ (usually Maximal Control Invariant Set), which ensures states remain feasible indefinitely under **Safe Control Law** $\pi_S(x)$

7.3 Implementation Types

Switching SF

Binary choice. If u_L leads to a safe state, use it. Otherwise, *fall back* to the safe input Kx. This can lead to *harsh* interventions.

Minimally Invasive SF

Formulated as a projection problem:

$$\min_{u_0} \|u_0 - u_L(k)\| \quad \text{s.t.} \quad f(x(k), u_0) \in \mathcal{S}, \quad u_0 \in \mathcal{U}$$

Control Barrier Function (CBF) SF

Introduces a *dampening* constraint to prevent the system from rushing toward the boundary of the safe set. It uses a parameter $\gamma \in (0, 1]$ to reduce the *remaining budget* until the boundary:

$$V(f(x, u_0)) - V(x) \leq \gamma(1 - V(x))$$

7.4 Model Predictive Safety Filter (MPSF)

While invariance-based filters are often restricted to small ellipsoidal sets (which are conservative), the **Predictive Safety Filter** uses MPC techniques to look further ahead.

Mechanism It plans a safe forward trajectory of N steps that terminates in a safe set $\mathcal{X}_f$.

The Formulation

$$\min_{u_0,\ldots,u_{N-1}} \|u_0 - u_L(k)\|$$

$$\text{s.t. } x_{i+1} = f(x_i, u_i), \quad (x_i, u_i) \in \mathcal{X} \times \mathcal{U}, \quad x_N \in \mathcal{X}_f$$

Benefit Any state that is *feasible* for this optimization is considered safe. This implicitly defines a much larger safe set $\mathcal{X}_{feas}$ than a simple ellipsoidal $\mathcal{S}$.

Robustness and Performance

Uncertainty In real-world applications (like the Quadrotor example), safety depends on a perfect model. To handle disturbances, the MPSF can be extended using **Robust Tube MPC** or **Stochastic MPC** techniques (e.g., constraint tightening).

Softening the Intervention To prevent the filter from *fighting* the learner, a barrier term can be added to the learner's own cost function to make it naturally prefer safe regions: $C_s(x, u) = C(x, u) - \gamma \log(1 - V(x))$

8 Iterative Learning Based MPC

In standard MPC, stability and recursive feasibility are guaranteed by terminal sets $\mathcal{X}_f$ and terminal costs l_f. IL-MPC constructs these *ingredients* dynamically from previous trajectories:

Sampled Safe Set ($\mathcal{SS}^e$) The collection of all successful state trajectories recorded up to episode e.

Convex Safe Set ($\mathcal{CS}^e$) The convex hull of SS^e. For linear systems, any convex combination of successful trajectories is also a feasible trajectory. This set is **control invariant**.

Learned Terminal Cost ($P^e(x)$) Defined as a barycentric function over the safe set. It assigns each point the minimum *cost-to-go* based on a convex combination of previous costs.

Guarantees Under linear dynamics, the resulting MPC is recursively feasible, asymptotically stable, and the closed-loop cost is non-increasing across episodes, eventually converging to the infinite-horizon optimal solution.

8.1 Global Tuning Approaches (Bayesian Methods)

Used when the relationship between cost parameters (e.g., Q, R, P) and a high-level performance metric (e.g., lap time) is unknown or non-analytical.

Bayesian Optimization (BO) Uses a **Gaussian Process** as a surrogate model to represent the unknown performance function. An **Acquisition Function** (like Lower Confidence Bound - LCB) suggests the next parameters to test by balancing exploration (uncertainty) and exploitation (predicted performance).

Safe BO Restricts exploration to parameters that are likely to maintain performance above a minimum safety threshold.

Contextual BO Adapts the tuning to changing environments or system parameters (e.g., racing different cars) by treating model mismatches as a *context* variable θ.

Bayesian MPC Uses **Thompson Sampling** to sample parameters from a posterior distribution and use them in the MPC, minimizing *cumulative regret*.

8.2 Local Tuning Approaches (Differentiable MPC)

If the performance metric is differentiable, we can use gradient-based methods.

Differentiable MPC Treats the entire MPC solver as a differentiable layer. This allows the computation of gradients of the optimal trajectories $\frac{\partial u^*}{\partial \theta}$ with respect to cost or dynamics parameters.

RL-MPC Uses Reinforcement Learning to update cost parameters θ to minimize long-term cost.

ZipMPC A student-teacher framework. A *Student* (short-horizon MPC) is trained via imitation learning to mimic the behavior of a *Teacher* (computationally expensive long-horizon MPC)

8.3 Approximate Explicit MPC

Standard MPC requires solving an optimization problem online, which may be too slow for high-frequency applications. Explicit MPC moves the computation offline.

Explicit MPC for Linear Systems

For linear systems with polyhedral constraints, the MPC solution is a **Multiparametric Quadratic Program (mp-QP)**.

The state space is partitioned into polyhedral regions. In each region, the control law is a unique **Piecewise Affine (PWA)** function: $\pi^*(x) = K_i x + k_i$.

Pros/Cons Online evaluation is just a *point-in-polyhedron* search (very fast), but the number of regions grows exponentially with constraints and horizon (high memory usage).

Approximate Explicit MPC for Nonlinear Systems

Nonlinear systems generally lack an analytical explicit solution. Instead, we learn a function approximation $\tilde{\pi}(x) \approx \pi^*(x)$.

Regression Techniques Neural Networks (NN) or Kernel Interpolation are used to fit a model to a dataset of offline-solved MPC problems.

Verification and Safety Because approximation errors can break stability/safety, two approaches are used:

1. **Safety Filters** Project the approximate input $\tilde{u}$ onto a control invariant set to ensure constraints are always met.
2. **Probabilistic Verification** Use the **Hoeffding Bound** to certify the controller's success rate μ with a specific confidence level $(1 - \delta_h)$ based on a finite number of test samples.

9 Invariance and Sets

Definition 1 (Positively Invariant Set $\mathcal{O}$)**.** For an autonomous or closed-loop system, the set $\mathcal{O}$ is positively invariant if:

$$x(k) \in \mathcal{O} \Rightarrow x(k+1) \in \mathcal{O}, \quad \forall k \in \{0, 1, \dots\}$$

Definition 2 (Maximal Positively Invariant Set $\mathcal{O}_\infty$)**.** A set that contains all $\mathcal{O}$ is the maximal positively invariant set $\mathcal{O}_\infty \subset \mathcal{X}$

Definition 3 (Pre-Sets)**.** The set of states that in the dynamic system $x(k+1) = g(x(k))$ in one time step evolves into the target set $\mathcal{S}$ is the **pre-set** of $\mathcal{S} \Rightarrow \text{pre}(\mathcal{S}) := \{x \mid g(x) \in \mathcal{S}\}$

Lemma 2. **Invariant Sets from Lyapunov Functions**

If $V : \mathbb{R}^n \to \mathbb{R}$ is a Lyapunov function for $x(k+1) = g(x(k))$, then $Y := \{x \mid V(x) \leq \alpha\}$ is an invariant set for all $\alpha \geq 0$

Proof. Lyapunov property $V(g(x)) - V(x) < 0$ implies that once $V(x(k)) \leq \alpha, V(x(j)) < \alpha, \forall\, j \geq k \to$ Invariance □

Example System $x(k+1) = Ax(k)$, $A^\top PA - P \prec 0 \prec P$ and resulting Lyapunov function $V(x(k)) = x(k)^\top Px(k)$

Goal Find the largest α s.t the invarinat set $Y_\alpha \in \mathcal{X}$

$$Y_\alpha := \{x \mid x^\top Px \leq \alpha\} \subset \mathcal{X} := \{x \mid Fx \leq f\}$$

Equivalent to $\max_\alpha \alpha$ s.t. $h_{Y_\alpha}(F_i) \leq f_i \; \forall i \in \{1 \dots n\}$

Theorem 6 (Geometric condition for invariance)**.** Set $\mathcal{O}$ is positively invariant set iff $\mathcal{O} \subseteq \text{pre}(\mathcal{O}) \Leftrightarrow \text{pre}(\mathcal{O}) \cap \mathcal{O} = \mathcal{O}$

Proof. **Necessary** if $\mathcal{O} \not\subseteq \text{pre}(\mathcal{O})$, then $\exists \bar{x} \in \mathcal{O}$ s.t $\bar{x} \notin \text{pre}(\mathcal{O}) \rightsquigarrow \bar{x} \in \mathcal{O}, \bar{x} \notin \text{pre}(\mathcal{O})$, thus $\mathcal{O}$ not positively invariant

Sufficient if $\mathcal{O}$ not positive invariant set, then $\exists \bar{x} \in \mathcal{O}$ s.t $g(\bar{x}) \notin \mathcal{O} \rightsquigarrow \bar{x} \in \mathcal{O}, \bar{x} \notin \text{pre}(\mathcal{O})$ thus $\mathcal{O} \notin \text{pre}(\mathcal{O})$ □

9.1 Control Invariance

Definition 4 (Control Invariant Set)**.** $\mathcal{C} \subseteq \mathcal{X}$ control invariant if

$$x(k) \in \mathcal{C} \Rightarrow \exists u(k) \in \mathcal{U} \text{ s.t } g(x(k), u(k)) \in \mathcal{C} \; \forall k$$

Definition 5 (Maximal Control Invariant Set $\mathcal{C}_\infty$)**.** A set that contains all $\mathcal{C}$ is the maximal control invariant set $\mathcal{C}_\infty \subset \mathcal{X}$

Intuition For all states in $\mathcal{C}_\infty$ exists control law s.t constraints are never violated $\rightsquigarrow$ **The best any controller could ever do**

Pre-set $\text{pre}(\mathcal{S}) := \{x \mid \exists u \in \mathcal{U} \text{ s.t } g(x, u) \in \mathcal{S}\}$

Set $\mathcal{C}$ is control invariant iff: $\mathcal{C} \subseteq \text{pre}(\mathcal{C}) \Leftrightarrow \text{pre}(\mathcal{C}) \cap \mathcal{C} = \mathcal{C}$

Control Law from Control Invariant Set

Control law $\kappa(x(k))$ will **guarantee** that the system with control invariant set $\mathcal{C}$ satisfies constraints **for all time** if

$$x(k+1) = g(x(k), u(k)) \to g(x, \kappa(x)) \in \mathcal{C} \; \forall x \in \mathcal{C}$$

We can use this fact to **synthesize** control law κ

$$\kappa(x) := \text{argmin}\{f(x, u) \mid g(x, u) \in \mathcal{C}\}$$

with f as any function (including $f(x, u) = 0$)

Does not ensure that system will converge
Difficult because calculating control invariant sets is hard
MPC implicitly describes $\mathcal{C}$ s.t easy to represent/compute

9.2 MPC Context

Often used as a terminal set $\mathcal{X}_f = \mathcal{C}_\infty$ to guarantee recursive feasibility in MPC formulations.

9.3 Real-World Notes

For nonlinear systems, computing $\mathcal{C}_\infty$ is NP-hard; approximations (e.g., ellipsoidal sets) are common. In robust MPC, extend to robust positively invariant (RPI) sets for disturbance handling.

9.4 Extensions and Comparisons

- Similar to robust invariant sets in RMPC (e.g., tightened via Pontryagin difference). - For stochastic MPC, consider probabilistic safe sets.

9.5 Robust Invariance

Definition 6 (Robust Positive Invariant Set $\mathcal{O}^\mathcal{W}$)**.** For the autonomous system $x(k+1) = f(x(k), w(k))$, the set $\mathcal{O}^\mathcal{W}$ is robust positive invariant if:

$$x \in \mathcal{O}^\mathcal{W} \Rightarrow f(x, w) \in \mathcal{O}^\mathcal{W}, \quad \forall w \in \mathcal{W}$$

Given set Ω and dynamic system $x(k+1) = f(x(k), w(k))$,

$$\text{pre}^\mathcal{W}(\Omega) := \{x \mid f(x, w)\} \in \Omega \; \forall w \in \mathcal{W}$$

Definition 7 (Robust Pre-Sets)**.** The set of states that in the dynamic system $x(k+1) = g(x(k), w(k))$ for all disturbance $w \in \mathcal{W}$ in one time step evolves into the target set Ω is the **pre-set** of $\Omega \Rightarrow \text{pre}^\mathcal{W}(\Omega) := \{x | g(x, w) \in \Omega \; \forall w \in \mathcal{W}\}$

Theorem 7 (Geometric condition for robust invariance)**.** Set $\mathcal{O}^\mathcal{W}$ is robust positive invariant iff $\mathcal{O}^\mathcal{W} \subseteq \text{pre}^\mathcal{W}(\mathcal{O}^\mathcal{W})$

Minimum Robust Invariant Set

$\mathcal{F}_\infty = \bigoplus_{j=0}^{\infty} A_K^j \mathcal{W}, \mathcal{F}_0 := \{0\} \Rightarrow \mathcal{F}_n = \mathcal{F}_{n+1} = \mathcal{F}_\infty$

9.6 Computing Invariant Sets and Pre-sets

System for **Pre-Set Computation**

$$\begin{aligned} x(k+1) &= Ax(k) + Bu(k) \\ u(k) \in \mathcal{U} &:= \{u | Gu \leq g\} \\ \mathcal{S} &:= \{x | Fx \leq f\} \end{aligned}$$

Invariant Pre-Set

$$\begin{aligned} \text{pre}(S) &:= \{x \mid Ax \in S\} \\ &= \{x \mid FAx \leq f\} \end{aligned}$$

Control Invariant Pre-Set

$$\begin{aligned} \text{pre}(S) &:= \{x \mid \exists u \in \mathcal{U}, Ax + Bu \in S\} \\ &= \{x \mid \exists u \in \mathcal{U}, FAx + FBu \leq f\} \\ &= \left\{x \mid \exists u \in \mathcal{U}, \begin{bmatrix} FA & FB \\ 0 & G \end{bmatrix} \begin{bmatrix} x \\ u \end{bmatrix} \leq \begin{bmatrix} f \\ g \end{bmatrix}\right\} \end{aligned}$$

This is a **projection** operation

System for **Robust Pre-Set Computation**

$$\begin{aligned} x(k+1) &= Ax(k) + w(k) \\ \Omega &:= \{x \mid Fx \leq f\} \end{aligned}$$

Robust Invariant Pre-Set

$$\begin{aligned} \text{pre}^\mathcal{W}(\Omega) &= \{x \mid FAx + Fw \leq f\} \\ &= \{x \mid FAx \leq f - \max_{w \in \mathcal{W}} Fw\} \\ &= \{x \mid FAx \leq f - h_{\mathcal{W}^i}(F)\} \end{aligned}$$

where $h_{\mathcal{W}^i}(F)$ is the **support function** $\sigma_\mathcal{C}(a) = \sup_{x \in \mathcal{C}} a^T x$